

TRABAJO PRÁCTICO INTEGRADOR

Alumnos - Comisión N°5

Khiara Yamil Razzolini Martinez

Franco Ian Rossi

**Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.
Organización Empresarial**

Docente Titular

Prof. Gabriela Martínez

Docentes Adjuntos

Prof. Carolina Bruno, Prof. Mario Raúl López, Prof. Andrea Ramos

19 de Junio de 2026

ÍNDICE

1. El negocio y el proceso elegido
2. Catálogo de productos
3. Reglas de negocio
4. Diagrama BPMN – As-Is
5. Diagrama BPMN – To-Be
6. Diccionario de datos
7. Máquina de estados
8. Desarrollo del bot en Node.js
9. Casos de camino infeliz probados
10. Manual de usuario
11. Consultas realizadas a herramientas de IA
12. Organización del repositorio
13. Conclusión

1. EL NEGOCIO Y EL PROCESO ELEGIDO

“Cometa” es un local de venta de hardware informático (procesadores, memorias RAM, placas de video, SSD, motherboards y fuentes) que puede analizarse como un sistema abierto: recibe entradas desde su entorno, las transforma mediante procesos internos y devuelve salidas que impactan en ese mismo entorno.

Entorno del sistema: clientes que necesitan información de stock y precios, proveedores que abastecen el inventario, y el mercado de hardware que define disponibilidad y precios de referencia.

Entradas: consultas de clientes (por teléfono o en el local) y el stock físico cargado por el local.

Componentes / subsistemas identificados:

Componente	Clasificación	Justificación
Vendedor	Humano / operativo	Ejecuta la consulta manual y decide sobre la marcha sin un criterio sistematizado.
Catálogo de productos	Información / dato maestro	Es la fuente de verdad sobre qué existe y a qué precio; si está desactualizado, todo el proceso falla.
Stock físico	Información / recurso	Determina si una venta puede concretarse; es el dato más volátil del sistema.
Planilla o memoria del vendedor	Soporte de información (informal)	Reemplaza a un registro formal; es el punto débil del sistema actual, porque no es confiable ni auditable.
Cliente	Humano / entorno	Genera la demanda que dispara el proceso.

Esta clasificación distingue entre componentes que producen información confiable (catálogo, stock) y componentes que dependen de la memoria o el criterio humano (vendedor, planilla informal), que es justamente donde se origina la ineficiencia.

Impacto en la gestión de información: Al no existir un repositorio único de consultas o reservas, el sistema no genera trazabilidad. Dos subsistemas pueden operar con información distinta sobre el mismo recurso (stock), lo que genera sobreventa o promesas que no se pueden cumplir. El proceso elegido (consulta de stock y reserva) es el punto de mayor fricción entre la información real del sistema y la que circula entre sus componentes.

2. CATÁLOGO DE PRODUCTOS

Los únicos seis productos que reconoce el bot. El motherboard se deja sin stock para probar ese caso particular sin inventar otro producto.

Código	Producto	Precio	Stock
CPU-R5-5600	Procesador AMD Ryzen 5 5600	\$185.000	8
CPU-I5-12400F	Procesador Intel Core i5-12400F	\$210.000	5
RAM-COR-16-D DR4	Memoria RAM Corsair Vengeance 16GB DDR4 3200MHz	\$42.000	15
GPU-RTX4060	Placa de video Gigabyte RTX 4060 8GB	\$520.000	3
SSD-KING-1TB	SSD Kingston NV2 1TB NVMe	\$68.000	20
MB-ASUS-B550 M	Motherboard Asus Prime B550M-A	\$110.000	0

Tabla 1. Catálogo de productos cargado en db.json

3. REGLAS DE NEGOCIO

Regla	Detalle
Catálogo cerrado	El bot solo reconoce los 6 productos del catálogo. Si el código no existe, avisa y pide reingresarlo (máximo 2 reintentos; al tercer error, corta y deriva a un vendedor).
Reserva automática	Si el stock disponible es mayor o igual a la cantidad pedida, el bot confirma la reserva y resta esa cantidad del stock en el JSON.
Stock insuficiente	Si el stock es menor a la cantidad pedida (incluido 0), el bot informa cuántas unidades hay disponibles y cierra la conversación sin reservar nada.
Vigencia de la reserva	Toda reserva confirmada queda pendiente de retiro por 48 horas (dato informativo; el vencimiento no está automatizado en el bot).

Tabla 2. Reglas de negocio del proceso automatizado

4. DIAGRAMA BPMN – AS-IS

Representa cómo se hace hoy la consulta de stock en Cometa, con dos carriles: Cliente y Vendedor. El cliente llama o va al local y pregunta si hay stock de un producto puntual. El vendedor revisa el depósito o una planilla que puede estar desactualizada. No queda ningún registro de la consulta ni de una reserva, por lo que dos clientes distintos pueden creer que reservaron el mismo producto.

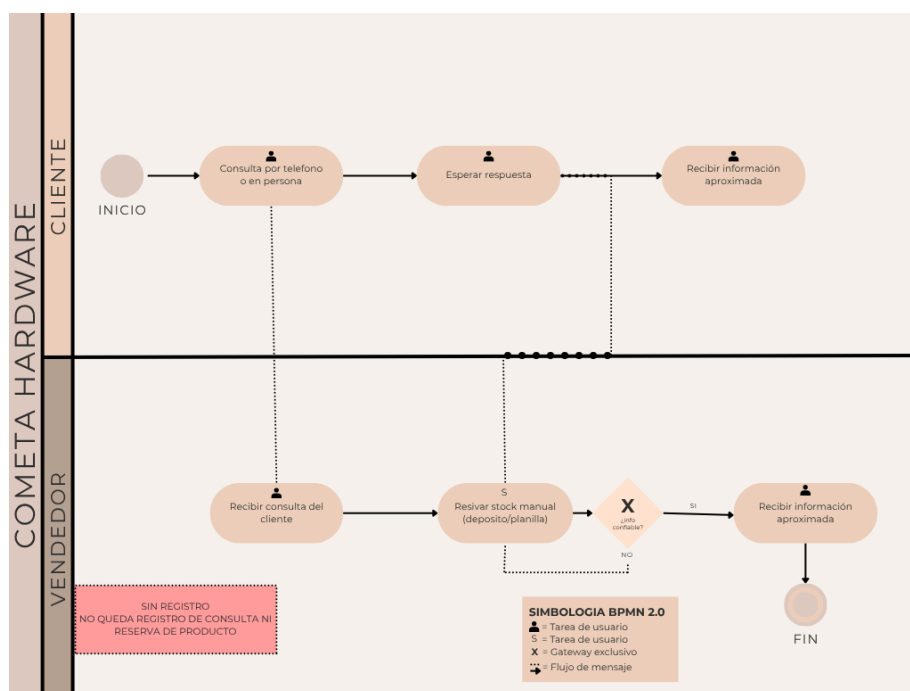


Figura 1. Diagrama BPMN As-Is del proceso actual en Cometa Hardware

5. DIAGRAMA BPMN – TO-BE

Representa el proceso automatizado con el bot, con dos carriles: Cliente/Usuario y Bot/Sistema. El evento de inicio es cuando el cliente le escribe al bot. Los eventos de fin posibles son: reserva confirmada, stock insuficiente, o derivación a un vendedor tras 2 reintentos fallidos.

Gateway 1: el código ingresado existe en el catálogo. **Gateway 2:** el stock disponible es mayor o igual a la cantidad pedida. Tareas de usuario: ingresar código y cantidad. Tareas de servicio: buscar el producto en el catálogo, calcular si el stock alcanza, descontar stock al confirmar.

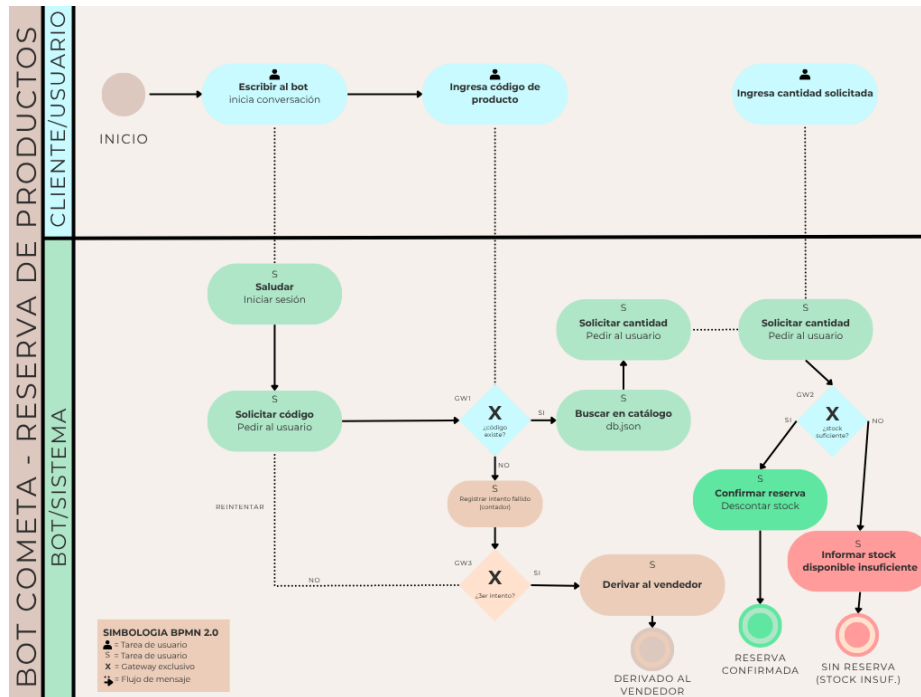


Figura 2. Diagrama BPMN To-Be del proceso con el bot

6. DICCIONARIO DE DATOS

Campo	Tipo	Descripción
producto.codigo	string	Identificador único del producto (ej. "GPU-RTX4060").
producto.nombre	string	Nombre comercial del producto.
producto.precio	number	Precio de venta en pesos.
producto.stock	number	Unidades disponibles actualmente.
reserva.cliente	string	Nombre simulado de quien hace la reserva.
reserva.producto_codigo	string	Código del producto reservado.
reserva.cantidad	number	Cantidad reservada.
reserva.estado	string	Valores posibles: "confirmada", "sin_stock_suficiente", "derivado".
reserva.stock_al_momento	number	Stock disponible en el momento de la consulta, antes de descontar. Permite auditar el estado del sistema al instante de cada reserva.

Tabla 3. Diccionario de datos del modelo producto/reserva

7. MÁQUINA DE ESTADOS

Tabla de transiciones implementada en el archivo estados.js del repositorio.

Estado actual	Evento	Estado siguiente
INICIO	El cliente le escribe al bot	ESPERANDO_CODIGO
ESPERANDO_CODIGO	Código existe en el catálogo	ESPERANDO_CANTIDAD
ESPERANDO_CODIGO	Código no existe (intento 1 o 2)	ESPERANDO_CODIGO (reintento)
ESPERANDO_CODIGO	Código no existe (3er intento)	FIN_DERIVADO
ESPERANDO_CANTIDAD	Stock disponible $\geq$ cantidad pedida	FIN_RESERVADO
ESPERANDO_CANTIDAD	Stock disponible $<$ cantidad pedida	FIN_SIN_STOCK

Tabla 4. Máquina de estados de la conversación

8. DESARROLLO DEL BOT EN NODE.JS

Se evaluaron tres alternativas: Telegram API, WhatsApp Business API y simulación por consola en Node.js. Se descartó WhatsApp por requerir cuenta de proveedor verificado. Telegram era viable, pero se priorizó la simulación por terminal para concentrar el esfuerzo en la lógica de negocio —máquina de estados, validaciones, persistencia— sin agregar la complejidad de webhooks y despliegue de servidor.

Se eligió Node.js sobre Python por su manejo simple de I/O con **readline-sync** y por facilitar una migración futura a Telegram API o backend web sin cambiar de stack. El proyecto se estructura en cuatro archivos: **index.js** (loop principal con los dos gateways), **estados.js** (tabla de transiciones), **db.js** (funciones **leerDB()** y **guardarDB()** para persistir el stock) y **db.json** (catálogo de productos y registro de reservas). El flujo fue probado de punta a punta, confirmando que las reservas se registran y el stock se descuenta correctamente.

9. CASOS DE CAMINO INFELIZ PROBADOS

Se probaron los cinco casos de camino infeliz exigidos por la consigna, validando que el bot responda de forma correcta ante entradas inválidas o situaciones de stock insuficiente.

Caso	Entrada de prueba	Comportamiento esperado
Código inexistente	GPU-RTX5090 (no está en el catálogo)	El bot avisa y pide reingresar, hasta 2 reintentos antes de derivar.
Cantidad con letras	"diez" en lugar de 10	El bot pide que se ingrese solo un número.
Cantidad negativa o cero	-3 o 0	El bot rechaza y vuelve a pedir la cantidad.
Más unidades de las disponibles	50 unidades de GPU-RTX4060 (stock real: 3)	El bot informa que solo hay 3 disponibles.
Producto sin stock	MB-ASUS-B550M (stock: 0)	El bot informa que no hay stock, sin pedir la cantidad.

Tabla 5. Casos de camino infeliz y comportamiento esperado

```
khiar@Kiki MINGW64 /d/Documentos/Universidad/PrimerAño/Organizacion-Empresarial/TPI/cometa-bot (main)
$ node index.js
Bot: Hola, bienvenido a Cometa.
Bot: Decime tu nombre, por favor.
Cliente: Khiara
Bot: Decime el codigo del producto que quieres consultar.
Cliente: GPU-RTX5090
Bot: Ese codigo no existe en nuestro catalogo. Intentalo de nuevo (te quedan 2 intentos).
Bot: Decime el codigo del producto que quieres consultar.
Cliente: GPU-RTX4060
Bot: Placa de video Gigabyte RTX 4060 8GB - $520.000.
Bot: Cuantas unidades quieres reservar?
Cliente: diez
Bot: Ingresar solo un numero, por favor.
Cliente: -8
Bot: La cantidad tiene que ser un numero entero mayor a cero.
Cliente: 50
Bot: No tenemos esa cantidad disponible. Actualmente hay 3 unidades en stock.
```

Figura 3. Captura de terminal — prueba de camino infeliz (repositorio: /capturas)

10. MANUAL DE USUARIO

A continuación, un ejemplo de conversación típica entre un cliente y el bot, incluyendo un caso de reserva exitosa y un caso de producto sin stock.

```

$ node index.js
Bot: Hola, bienvenido a Cometa.
Bot: Decime tu nombre, por favor.
Cliente: Franco
Bot: Decime el codigo del producto que quieres consultar.
Cliente: RAM-COR-16-DDR4
Bot: Memoria RAM Corsair Vengeance 16GB DDR4 – $42.000.
Bot: Cuantas unidades quieres reservar?
Cliente: 2
Bot: Hay stock suficiente. Reserva confirmada: 2 unidades. Stock restante: 9.

```

Figura 4. Captura de terminal — reserva exitosa (repositorio: /capturas)

Toda reserva confirmada queda pendiente de retiro durante 48 horas. Pasado ese plazo sin que el cliente la retire, el local puede liberar el producto nuevamente (vencimiento informativo, no automatizado en el bot).

11. CONSULTAS REALIZADAS A HERRAMIENTAS DE IA

Se utilizó **Gemini** para las consultas de arquitectura y código (ver carpeta /capturas del repositorio) porque permite iterar rápido sobre fragmentos de código y dar respuestas concretas en formato JSON/JavaScript. Los prompts se enfocaron en pedir estructuras de datos y patrones de implementación, evitando pedir código completo para forzar la comprensión propia de la lógica antes de escribirla.

Se utilizó **NotebookLM** para las consultas conceptuales sobre BPMN (ver carpeta /capturas del repositorio) porque permite cargar el material de cátedra como fuente y obtener respuestas ancladas a esos apuntes. Los prompts apuntaron a resolver dudas puntuales de notación (tipos de compuertas, cuándo usar un subprocesso) que surgieron al momento de diagramar el To-Be.

En ambos casos las respuestas de la IA se usaron como guía de criterio y no se trasladaron literalmente: el diagrama BPMN final y el código del bot fueron adaptados al catálogo de productos y a las reglas de negocio específicas de Cometa.



Figura 5. Ejemplo de consulta a Gemini durante el desarrollo (ver carpeta /capturas del repositorio)

12. ORGANIZACIÓN DEL REPOSITORIO

El repositorio en GitHub se organizó de la siguiente manera:

index.js, estados.js, db.js, db.json: código fuente del bot.

/docs: diagramas BPMN As-Is y To-Be exportados en alta resolución.

/capturas: pruebas del camino infeliz y consultas a la herramienta de IA.

README.md: descripción del proceso, instrucciones de instalación (**npm install**) y ejecución (**node index.js**), y estructura de carpetas del proyecto.

13. CONCLUSIÓN

El proceso de consulta y reserva en "Cometa" carecía de un mecanismo que sincronizara el stock real con la información que circulaba entre vendedor y cliente. Esa divergencia era el origen de la sobreventa y las respuestas inconsistentes.

El bot resuelve eso con una única fuente de verdad: cada consulta se valida contra db.json y cada reserva confirmada lo modifica de inmediato. El vendedor queda liberado del trabajo repetitivo y solo interviene en casos derivados; el cliente recibe siempre una respuesta consistente. Además, la organización pasa a contar con un registro que antes no existía: qué se consultó, en qué cantidad, y con qué resultado. Esa es la diferencia entre un proceso y un sistema.